

title: Fixed-point vs floating-point — platform survey and WF removal cost date: 2026-05-10 status: investigation

Fixed-point vs floating-point: platform survey and WF removal cost

WF is built around a `Scalar` abstraction (`wfsource/source/math/scalar.hp`) that compiles three ways:

- `SCALAR_TYPE_FIXED` — 16.16 fixed-point (PS1-era target)
- `SCALAR_TYPE_FLOAT` — IEEE 754 single
- `SCALAR_TYPE_DOUBLE` — IEEE 754 double

The Linux build currently uses `SCALAR_TYPE_FLOAT` (`engine/build_game.sh:128`). The fixed-point path still compiles and gates code in ~10 `.cc/.hp` files. This document answers: what FPU support exists across mainline platforms, what we lose by dropping fixed point, and what removal looks like in practice.

1. Floating-point support across mainline computers and consoles

Desktop CPUs

Era	Family	FPU
1980s	x86 (8086–80386)	optional 8087/80287/80387 coprocessor
1989+	80486DX	on-die x87 (DX); 486SX had none
1993+	Pentium and later	x87 + later SSE/SSE2/AVX
1985+	68020 + 68881/2	optional coprocessor
1990+	68040/68060	integrated (LC variants lacked it)
1995+	PowerPC	integrated
2005+	x86-64	SSE2 mandatory (scalar FP via SSE)
ARM desktop (Apple Silicon, Windows-on-ARM)	NEON + VFP mandatory in AArch64	

By 1995 every shipping desktop CPU had hardware floating point. Today every desktop / laptop / phone has both scalar and SIMD FP.

Home consoles (last-without-FPU bolded)

Console	Year	CPU	FPU?
Atari 2600	1977	6507	no
NES	1983	6502	no
Sega Master System	1985	Z80	no
Genesis / Mega Drive	1988	68000	no
TurboGrafx-16	1987	HuC6280	no
SNES	1990	65C816	no

Console	Year	CPU	FPU?
Neo Geo	1990	68000	no
3DO	1993	ARM60	no
Atari Jaguar	1993	68000 + Tom/Jerry	no (last Atari home console)
Sega Saturn	1994	dual SH-2	no (last Sega home console without FPU)
Sony PlayStation	1994	R3000A + GTE	no on CPU — GTE is a fixed-point geometry coprocessor (last Sony home console without FPU)
Nintendo 64	1996	MIPS R4300i	yes (paired-single-style FPU)
Dreamcast	1998	SH-4	yes (single + SIMD FV4)
PS2	2000	Emotion Engine + VU0/VU1	yes
GameCube	2001	Gekko (PowerPC)	yes
Xbox	2001	x86 (Coppermine)	yes
Everything since (Wii, 360, PS3, Wii U, PS4, XBO, Switch, PS5, XSX, Switch 2)	—	—	yes

Last home console without FPU per lineage:

- **Sony** — PlayStation 1 (1994). The GTE coprocessor is fixed-point, but the R3000A CPU itself has no FPU.
- **Sega** — Saturn (1994).
- **Nintendo** — SNES (1990). N64 added an FPU.
- **Atari** — Jaguar (1993). Atari exited the console market afterward.
- **Microsoft** — never shipped one; the original Xbox debuted in 2001 with x86 (Pentium III-class) + x87/SSE.

Handhelds

Handheld	Year	CPU	FPU?
Game Boy	1989	Sharp LR35902	no
Game Boy Color	1998	LR35902	no
Game Boy Advance	2001	ARM7TDMI	no
Nintendo DS	2004	ARM9 + ARM7	no (last Nintendo handheld without FPU)
PSP	2004	MIPS Allegrex + VFPU	yes
3DS	2011	ARM11 (VFPv2)	yes
PS Vita	2011	Cortex-A9 (VFPv3 + NEON)	yes
Switch / Switch 2	2017 / 2024	ARM Cortex-A / Ampere	yes
Steam Deck	2022	x86-64 (Zen 2)	yes

Last handheld without FPU per lineage:

- **Nintendo** — DS (2004). 3DS added VFPv2.
- **Sony** — never shipped one; PSP had a VFPU from launch.
- **Sega Game Gear** / **Atari Lynx** / **NGPC** — none had FPUs, all defunct lineages.

Smartphones

Numerically the biggest category and the most boring for this survey: **every smartphone ever shipped has had a hardware FPU.**

Era	Representative phones	CPU / FPU
2007	iPhone (Samsung S5L8900)	ARM11, VFPv2 single-precision
2008–2009	HTC Dream (G1), iPhone 3G	ARM11 + VFP
2009–2011	iPhone 3GS / 4, Galaxy S, Nexus One	Cortex-A8 / A9 , VFPv3 + NEON
2012–2014	iPhone 5 / 5S, Galaxy S3 / S4 / S5	Cortex-A15 / Apple A7, NEON + VFP; A7 is the first 64-bit phone SoC
2014+	All AArch64 phones (every iPhone since 5S, every Android flagship since 2014)	ARMv8-A with mandatory NEON + double-precision FP
2020+	iPhone 12+, Pixel 6+, Galaxy S22+	ARMv8.4-A / ARMv9, NEON + SVE2 on newer Snapdragons

Every Android device on the Play Store since the AArch64 cutoff (Google's 64-bit-only requirement, fully enforced 2021) has full hardware FP. iOS has been AArch64-only since iOS 11 (2017). For WF this is essentially the same conclusion as desktops: nothing to do — `SCALAR_TYPE_FLOAT` works everywhere.

The only phone-shaped device in recent memory that *lacked* a phone-class CPU was the [KaiOS](#) feature-phone revival on Snapdragon 205 (Cortex-A7 cores with VFPv4) — still has an FPU. Even the cheapest "dumb smartphones" sold today are ARMv7/v8 with FP.

Streaming devices and smart TVs

Same conclusion as phones — every shipping device has a hardware FPU — but worth documenting because the lineage spans different SoC vendors and the dongle-class hardware is sometimes mistaken for "small enough to lack an FPU."

Year	Device	SoC	CPU / FPU
2013	Chromecast (1st gen)	Marvell DE3005	single Cortex-A9, VFPv3 + NEON
2015	Chromecast (2nd gen) / Chromecast Audio	Marvell Armada 1500 Mini Plus	dual Cortex-A7, VFPv4 + NEON
2016	Chromecast Ultra	Marvell Armada 1500 Mini Plus 88DE3006	dual Cortex-A53, ARMv8-A NEON
2020	Chromecast with Google TV (4K)	Amlogic S905X3 / S905D3	quad Cortex-A55, ARMv8.2-A NEON
2022	Chromecast with Google TV (HD)	Amlogic S805X2	quad Cortex-A35, ARMv8-A NEON
2024	Google TV Streamer	MediaTek MT8696	quad Cortex-A55, 4 GB RAM
2015+	Sony / TCL / Hisense Android TV / Google TV sets	MediaTek MT58xx / Amlogic	Cortex-A53/A55/A73, ARMv8-A NEON
2007+	Apple TV (since 2nd gen, A4-onwards)	Apple A-series	ARMv7-A then ARMv8-A, NEON + VFP
2014+	Fire TV	MediaTek / Amlogic	Cortex-A53/A55, ARMv8-A NEON
2017+	NVIDIA Shield TV	Tegra X1	quad Cortex-A57 + quad A53, ARMv8-A NEON + double-precision FP

The cheapest, most RAM-constrained device in the table — the 2022 [Chromecast with Google TV HD](#) at 1.5 GB RAM, \$30 — still has four Cortex-A35 cores with full ARMv8-A NEON. For the fixed-point question this category is a no-op: `SCALAR_TYPE_FLOAT` works on every streaming dongle Google, Apple, Amazon, Roku, or NVIDIA has ever shipped.

2. Embedded SoC and MCU floating-point

Floating-point coverage on small parts is much more uneven than on mainline platforms. Rough survey:

No FPU (software float only)

- **AVR 8-bit** — every Arduino classic (Uno, Nano, Mega, Leonardo, Micro). Float costs ~thousands of cycles per op.
- **PIC8/PIC16/PIC18, MSP430** — entire lineage.
- **Cortex-M0 / M0+** — Raspberry Pi Pico (RP2040), STM32 F0/L0/G0, SAMD21 (Arduino Zero), nRF51.
- **Cortex-M3** — STM32 F1/F2/L1, SAM3X8E (Arduino Due), nRF52810.
- **ESP8266** (Tensilica L106) — no FPU.
- **ESP32-S2** (Xtensa LX7) — the LX7 core's FPU is *optional*; S2 ships without it.
- **ESP32-C2 / C3 / C5 / C6 / C61 / H2** — every RISC-V ESP32 to date uses an ISA string with **no F extension** (RV32IMC / RV32IMAC). No hardware FP.
- **GBA/DS retro targets** (ARM7TDMI, ARM9) — same situation, just for hobbyist ports.

Hardware single-precision FPU

- **Cortex-M4F** — STM32 F3/F4/L4, nRF52832/40, SAMD51, RP2350 (M33F).
- **Cortex-M7** — STM32 F7/H7, Teensy 4.x (IMXRT1062), portenta.
- **ESP32 (Xtensa LX6) and ESP32-S3 (LX7-with-FPU)** — single-precision FPU; doubles are software. (S2 is the odd one out — same LX7 core as S3 but with the FPU left off.)
- **ESP32-P4** (RV32IMAFC) — the first Espressif RISC-V part with hardware FP; single precision.
- **Other RISC-V with F extension** — BL808, K210, JH7110 (VisionFive 2).
- **Most Cortex-A class SoCs** (Raspberry Pi 3/4/5, Allwinner, Rockchip, Jetson, BeagleBone) — full NEON + VFP.

Hardware double-precision FPU

Almost exclusively Cortex-A (Linux-capable) and high-end Cortex-M7 with `DP` variant. Doubles on Cortex-M4F and ESP32 are emulated.

Rule of thumb: anything that runs Linux has full FP; anything you flash a `.hex` to is a coin-flip; anything 8-bit or sub-Cortex-M4 has none.

The ESP32 family in detail

Espressif's ESP32 line is the most relevant embedded ecosystem for a project like WF — cheap, widely available, hobbyist-friendly, and several variants have enough headroom (with external PSRAM) for a 2 MB-class engine. There are now 10 distinct ESP32 SoCs shipping; only **three** of them have *both* an FPU and a PSRAM path big enough for WF.

SoC	Year	Core	FPU	SRAM	PSRAM	Wireless	WF?
ESP32 classic	2016	LX6 ×2 @240	Single	520 KB	8 MB	Wi-Fi 4, BT 4.2	✓
ESP32-S2	2020	LX7 ×1 @240	—	320 KB	8 MB	Wi-Fi 4	✗
ESP32-S3	2021	LX7 ×2 @240	Single + SIMD	512 KB	32 MB	Wi-Fi 4, BLE 5	✓
ESP32-C2	2022	RV32IMC ×1 @120	—	272 KB	—	Wi-Fi 4, BLE 5	✗
ESP32-C3	2020	RV32IMC ×1 @160	—	400 KB	—	Wi-Fi 4, BLE 5	✗
ESP32-C5	2024	RV32IMAC ×2 @240	—	384 KB	—	Wi-Fi 6 dual, BLE 5	✗
ESP32-C6	2023	RV32IMAC ×1 @160	—	512 KB	—	Wi-Fi 6, BLE 5, 802.15.4	✗
ESP32-C61	2024	RV32IMAC ×1 @160	—	320 KB	—	Wi-Fi 6, BLE 5	✗
ESP32-H2	2023	RV32IMAC ×1 @96	—	320 KB	—	BLE 5, 802.15.4	✗
ESP32-P4	2024	RV32IMAFC ×2 @400	Single	768 KB	32 MB	— (companion chip)	✓

Clock speeds in MHz. "Cores $\times$ N @MHz" means N cores at that clock. PSRAM column is max external; "—" means no PSRAM bus on the chip. WF needs both an FPU and a ≥ 2 MB PSRAM path — only classic, S3, and P4 satisfy both. The C-series and H2 are all RISC-V variants whose ISA string omits the `F` extension, hence no hardware single-precision FP.

[!IMPORTANT]

Only **three** ESP32 SoCs are viable WF hosts as of 2026: [ESP32 classic](#), [ESP32-S3](#), and [ESP32-P4](#). Of these, **ESP32-S3** is the practical recommendation — abundant dev boards ([LOLIN S3](#), [M5Stack CoreS3](#), [LilyGO T-Display S3](#), etc.), mature [ESP-IDF](#) support, 8 MB PSRAM is standard on most modules, and the integrated BLE radio matters for controllers. **ESP32-P4** is the future-looking option for richer scenes once dev boards ([ESP32-P4 Function EV Board](#), [M5Stack Tab5](#)) mature — it has a dedicated 2D GPU and MIPI-DSI for higher resolutions, but lacks on-chip Wi-Fi/BT and needs a companion radio chip.

The seven non-viable variants are blocked by missing FPU (every C-series and H2), missing PSRAM (every C-series and H2), or both. None of them gain anything from WF keeping `SCALAR_TYPE_FIXED`, because **they don't have the RAM either** — fixed-point math doesn't make a 400 KB SRAM chip fit a 2 MB engine.

3. What platforms does keeping fixed-point support enable for WF?

The interesting question isn't "are these reachable in theory" but "is fixed-point support load-bearing for them." The categories:

Genuinely require fixed-point for usable performance — and how the RAM budget shakes out (PS1 reference: 2 MB main + 1 MB VRAM + 512 KB sound):

- **AVR Arduinos** (Uno: 2 KB SRAM, Mega: 8 KB). RAM disqualifies regardless of FP.
- **SAMD21** (Arduino Zero): 32 KB SRAM, no external RAM bus. Disqualified.
- **RP2040** (Pi Pico, Cortex-M0+): 264 KB SRAM, no PSRAM support. Doesn't fit 2 MB.
- **ESP32-C2 / C3 / C5 / C6 / C61 / H2** (RISC-V no-F): 272–512 KB SRAM, no PSRAM path. Doesn't fit, and fixed-point wouldn't help.
- **ESP32-S2** (LX7 no-FPU): 320 KB SRAM + up to 8 MB octal PSRAM. **Fixed-point would unlock this — and only this — across the ESP32 family.** It's the one Espressif chip with the FPU left off but PSRAM still available.
- **RP2350-without-FPU** (Cortex-M33, FPU is optional): 520 KB SRAM + **external PSRAM** over QSPI → 2 MB+ reachable. Other genuine candidate. Configurations with the FPU enabled obviously prefer float.

For reference, ESP32/MCU parts with enough RAM *and* a hardware FPU (no fixed-point needed):

- **ESP32 classic** (LX6): 520 KB SRAM + 8 MB PSRAM, single FPU.
- **ESP32-S3** (LX7-with-FPU): 512 KB SRAM + 32 MB octal PSRAM, single FPU. Recommended target.
- **ESP32-P4** (RV32IMAFC): 768 KB SRAM + 32 MB octal PSRAM, single FPU, 2D GPU, MIPI-DSI.
- **Teensy 4.x** (Cortex-M7): 1 MB SRAM + optional 8 MB PSRAM, single + double FPU.

The original WF target was PS1 (2 MB, no CPU FPU, GTE coprocessor for fixed-point geometry). That target is gone. Today's only "fixed-point unlocks an extra platform" candidates are **ESP32-S2** (no FPU + 8 MB PSRAM) and **RP2350 with the FPU disabled** — niche choices in both cases.

Helped by fixed-point but not blocked:

- ESP32 classic / S3 / P4 — all have FPUs and ample PSRAM. WF would use singles. Fixed-point not needed.
- Cortex-M7 boards (Teensy 4, Portenta H7) have FPUs. Not needed.
- Old retro targets (PS1, Saturn, GBA, DS) — these are the *original* reason fixed-point exists, and they remain the only realistic platforms where dropping fixed-point removes meaningful capability.

Not helped at all (already have FPUs):

- Every console from N64 / Dreamcast forward.
- Every PSP/3DS/Vita/Switch/Steam Deck.
- Every desktop, laptop, phone, tablet, Pi, Jetson, modern smart TV.

Conclusion: in 2026, keeping `SCALAR_TYPE_FIXED` is essentially a *retro-target option* — PS1/Saturn/GBA/DS homebrew ports. It is not a gate for any modern shipping platform.

4. Removing fixed-point from WF — cost survey

What's there now

- `wfsource/source/math/scalar.hp` — defines `Scalar` as fixed or float behind macros.
- `wfsource/source/math/linux/scalar.hpi` + `scalar.cc` — fixed-point inline implementations and assembly fallbacks.
- 16.16 internal representation; `SCALAR_ONE_LS = 1<<16`.
- ~10 `.cc/.hp` files contain `#ifdef SCALAR_TYPE_FIXED` branches (`scalar`, `vector2`, `vector3`, `matrix34`, `angle`, `level.cc`, `main.cc`, `mathtest`).
- `floatscalar.hpi` already exists as the float counterpart.
- `Angle` stores 16-bit revolutions and converts via `Scalar(0, _value)` — `angle.hpi` has fixed-only inlines that vanish under float builds.

What removing it gains

- Delete ~200 lines of platform asm in `scalar.cc` and dead branches across `vector2.cc`, `vector3.cc`, `matrix34.hpi`, `mathtest.cc`, `level.cc`, `main.cc`.
- One fewer build axis (current matrix already only uses float on Linux/Android/iOS — fixed-point is no longer exercised by CI).
- Removes a class of latent bugs from overflow in 16.16 arithmetic that no shipping target sees.
- `Scalar` becomes a thin typedef / wrapper around `float`. Eventually it can be deleted outright and replaced with `float`, but that's a much bigger refactor.

What removing it costs

- Loses the ability to retarget PS1/Saturn/GBA/DS without writing a fixed-point math layer from scratch.
- The fixed-point `Angle` representation (16-bit revolutions stored as `uint16`) is *also* the on-disk and on-the-wire representation. That has to stay regardless of `Scalar`'s internal type.
- Any oracle-byte-identical test that round-trips a level through fixed-point `Scalar` arithmetic would need to be re-baselined.
- Loses **exact equality comparison**. Fixed-point `Scalar` values compare with `==` deterministically — same input bits, same answer, regardless of how each side was computed. Under `SCALAR_TYPE_FLOAT` the same sites become hazardous: arithmetic results rarely round-trip exactly, and NaN never compares equal to anything including itself. Every existing `Scalar a == b` (and `!=`) site would need audit; today they're scattered across animation, asset, baseobject, anim-channel and dozens of other subsystems. Mitigation: overload `Scalar::operator== / !=` with an epsilon-based comparison so call sites keep reading naturally — the underlying float still has the precision pitfalls but the *site-level* semantics stay close to the fixed-point version. Picking the right epsilon is itself a per-context judgement call (world-space positions vs. normalised weights need very different tolerances), so this isn't a free swap.

Storage: on-disk fixed-point doesn't save bytes at the same width

The `Scalar` runtime type and the `.iff` on-disk representation are independent decisions. Today:

- Mailboxes are fixed-point on real target, float on PC dev (`project_mailboxes_fixed_point` memory).
- Angles are revolutions in `uint16` — already a fixed-point format on disk and intentional, and *narrower* than a float, so an actual size win.

[!IMPORTANT]

`Vector3` fields in OAS/asset chunks are stored as `Scalar` payload, which under `SCALAR_TYPE_FLOAT` is a 32-bit IEEE float — not the original fixed-point on-disk format. So WF on PC has already silently moved away from the PS1-era 16.16 disk encoding; the disk layout drifted with the runtime type. Anyone reading a current PC-built `cd.iff` expecting 16.16 in those fields will get floats instead.

There is no disk-size argument for keeping 16.16 fixed-point storage: 16.16 is 32 bits, IEEE float is 32 bits, same bytes per value. The only on-disk reasons to use fixed-point are:

1. **Going narrower than 32 bits** — an 8.8 or 4.12 format halves storage at the cost of range/precision. Useful for known-bounded values (normalized vectors, per-vertex weights, scales near 1.0), not for general world-space positions where range matters.
2. **Bit-exact reproducibility on a fixed-point target** — irrelevant once `SCALAR_TYPE_FIXED` is gone.

The `Angle uint16` (revolutions in 16 bits) is a good example of #1 — 16 bits is half a float, and revolutions are inherently bounded `[0, 1)` so the narrow range is fine. There's no equivalent argument for full-range `Scalar` fields.

Proposed phasing

1. **Remove `SCALAR_TYPE_FIXED` runtime build mode.** Delete the platform asm, simplify `scalar.hp/hpi` to "Scalar wraps float (or double)." Keep `SCALAR_TYPE_FLOAT / SCALAR_TYPE_DOUBLE` for now — doubles still useful on desktop for testing precision regressions.
2. **Collapse the typedef.** Replace `Scalar` with `float` everywhere it's purely numeric; keep `Angle` as its own type because its on-disk encoding is a revolution-uint16.

Step 1 is mechanical and safe; step 2 is a wide rename. Narrow-fixed-point storage formats (8.8, 4.12) for specific known-bounded fields would be a separate, much narrower project — argued on precision-per-byte, not on "saving fixed-point."

5. Recommendation

Drop `SCALAR_TYPE_FIXED` as a runtime option. It guards code paths that are not exercised by any current or planned WF target (Linux desktop, Android, iOS — all have hardware FPUs). The platforms it would still enable are:

- Retro targets (PS1, Saturn, GBA, DS) — feasible RAM-wise (PS1 itself had 2 MB), but each needs its own GPU/audio shim, and that work dwarfs the math-layer cost either way.
- **ESP32-S2** — the one ESP32 SoC with PSRAM but no FPU.
- **RP2350 with the FPU left off** — Cortex-M33 + external PSRAM.

None of these is on any roadmap. The realistic ESP32 targets (classic, S3, P4) all have FPUs and don't need fixed-point. Anyone seriously porting to a no-FPU target would be doing a much larger platform-bringup project, of which the math layer is a small part — and they can just as easily implement `Scalar` on top of a fixed-point library at that point, without us carrying the option in mainline.

No disk-size argument keeps fixed-point alive: 16.16 is the same 32 bits as float. Narrow fixed-point storage (8.8, 4.12) for bounded fields is a separate, much smaller project — argued on precision-per-byte, not on rescuing `SCALAR_TYPE_FIXED`.

Sources

Mainline CPUs & consoles

- ARM Cortex-M reference manuals — FPU presence per part: [ARM Cortex-M comparison](#)
- Console CPU specs cross-referenced against Copetti's "Architecture of Consoles" series for SNES, PS1, Saturn, N64, Dreamcast, PS2, GameCube, GBA, DS, 3DS, Vita.
- PlayStation GTE as a fixed-point geometry coprocessor (not a general FPU): [PS1 architecture writeup](#).

ESP32 family

- Espressif product overview: [ESP product selector](#) and per-chip pages under www.espressif.com/en/products/socs.
- [Wikipedia: ESP32](#) — variant table with ISA strings (RV32IMC vs RV32IMAFIC), FPU presence per SoC, core counts and clock speeds.
- ESP32-S2 datasheet (v1.8): [esp32-s2_datasheet_en.pdf](#) — confirms Xtensa LX7 single-core, 320 KB SRAM, optional external flash/PSRAM, no FPU in CPU & Memory section.
- ESP32-S3 datasheet: [esp32-s3_datasheet_en.pdf](#) — dual LX7 with FPU + SIMD, 512 KB SRAM, octal PSRAM support.
- ESP32-P4 datasheet: [esp32-p4_datasheet_en.pdf](#) — RV32IMAFIC dual-core HP @ 400 MHz, single FPU, 768 KB on-chip SRAM, up to 32 MB octal PSRAM, 2D pixel engine, MIPI-DSI/CSI.
- ESP32-C6 datasheet: [esp32-c6_datasheet_en.pdf](#) — RV32IMAC, no F extension.
- ESP32-C5 product page: www.espressif.com/en/products/socs/esp32-c5 — Wi-Fi 6 dual-band, RV32IMAC.